RAPPORT DU PROJET DE PYTHON

Application Système de Gestion de Bibliothèque

Réalisé par : **NAIMI Zineb**

Filière : Génie Informatique – GI3

Encadrant : ZAKARIA HAJA

Année universitaire : 2024 / 2025

Sommaire :

I. Introduction: **3**
1. Présentation du projet: 3
2. Objectifs du Projet: 3
II. Outils et technologies utilisés : **3**
III. Analyse et conception: **4**
1. Architecture du projet: 4
2. Diagramme de classes UML: 4
IV. Explication des algorithmes clés : **6**
1. Gestion des emprunts et retours: 6
Objectif : 6
2. Sauvegarde et chargement des données: 7
Objectif : 7
Exemple : sauvegarde des livres 7
3. Génération des visualisations statistiques: 7
Objectif : 7
Exemple : top 10 auteurs 7
V. Visualisation des données: **7**
1. Répartition des livres par genre: 7
2. Top 10 des auteurs les plus empruntés: 8
3. Emprunts des 30 derniers jours: 8
VI. Interface graphique: **8**
Captures d'écran: 9
VII. Difficultés rencontrées et solutions apportées: **10**
VIII. Conclusion : **11**

I. Introduction:

1. Présentation du projet:

Le projet **Gestion Bibliothèque** est une application de bureau développée en Python avec **Tkinter** pour l'interface graphique et des bibliothèques comme **matplotlib** et **pandas** pour la visualisation des données. Ce logiciel a été conçu pour répondre aux besoins fondamentaux d'une bibliothèque moderne en matière de gestion des livres, suivi des emprunts, gestion des membres, ainsi que l'affichage statistique des activités.

L'application offre une interface conviviale permettant aux bibliothécaires d'ajouter, modifier ou supprimer des livres et des membres, tout en assurant un suivi en temps réel des emprunts et des retours. Elle intègre également un système de sauvegarde et de chargement des données via des fichiers textes, assurant la persistance des informations entre les sessions.

Le système peut être utilisé aussi bien dans un environnement **console** (menu interactif) que dans une **interface graphique**, assurant une grande flexibilité pour différents types d'utilisateurs.

2. Objectifs du Projet:

- Gérer une bibliothèque de livres : ajouter, supprimer, lister
- Inscrire les membres de la bibliothèque
- Suivre les emprunts et les retours des livres
- Générer des statistiques sur les emprunts
- Sauvegarder et charger les données via des fichiers .txt et .csv:

II. Outils et technologies utilisés :

Langage de programmation :

- Python 3.13

Interfaces utilisateur :

- Tkinter (pour l'interface graphique)
- Terminal (interface en ligne de commande)

Visualisation des données :

- matplotlib
- pandas

Gestion des fichiers :

- Fichiers `.txt` pour les livres et membres

- Fichier `.csv` pour l'historique des emprunts

Environnement de développement :

- visual studio code

III. Analyse et conception:

1. Architecture du projet:

```
Gestion_Bibliotheque_NAIMI_Zineb/
├── data/                    # Données persistantes
│   ├── livres.txt           # Stockage des livres
│   ├── membres.txt          # Stockage des membres
│   └── historique.csv       # Historique des emprunts/retours
│
├── src/                     # Code source principal
│   ├── main.py              # programme principal
│   ├── bibliotheque.py      # Classes (Livre, Membre, Bibliotheque, etc.)
│   ├── exceptions.py        # Exceptions personnalisées
│   └── visualisations.py    # Interface utilisateur (Tkinter) + génération des graphiques  (matplotlib)
│
├── docs/                    # Documentation
│   └── rapport.pdf          # Rapport du projet
│
├── assets/                  # Ressources multimédia (images, vidéos)
│   ├── stats_genres.png     # Graphique : répartition des genres
│   ├── stats_auteurs.png    # Graphique : top auteurs
│   ├── stats_emprunts.png   # Graphique : emprunts des 30 derniers jours
│   └── presentation.mp4     # Vidéo de présentation du projet
│
├── requirements.txt         # Liste des bibliothèques Python nécessaires
├── README.md                # Présentation, installation et utilisation du projet
```

2. Diagramme de classes UML:

1. Classe `Bibliotheque` :

Attributs :

- `livres : List[Livre]` → Liste des livres disponibles dans la bibliothèque.
- `membres : List[Membre]` → Liste des membres inscrits.
- `historique : List[Tuple(date,id,titre)]` → Liste des enregistrements d'emprunts, chaque enregistrement contenant la date, l'identifiant du membre, et le titre du livre emprunté ou retourné.

Méthodes :

- `ajouter_livre(livre : Livre)` → Ajoute un nouveau livre au catalogue.
- `ajouter_membre(membre : Membre)` → Inscrit un nouveau membre.
- `emprunter_livre(id : int, titre : char)` → Gère l'emprunt d'un livre par un membre.
- `retourner_livre(id : int, titre : char)` → Gère le retour d'un livre.
- `top_10_auteurs()` → Fournit la liste des auteurs les plus empruntés.
- `genre_livres()` → Fournit une analyse par genre.

2. Classe `Membre`:

Attributs :

- `id_membre : int` → Identifiant unique du membre.
- `nom : char`→ Nom du membre.
- `livres_empruntes : List[Livre]` → Liste des livres actuellement empruntés.

Méthodes :

- `emprunter(livre : Livre)` → Permet au membre d'emprunter un livre.
- `retourner(livre : Livre)` → Permet au membre de retourner un livre.

3. Classe `Livre`

Attributs :

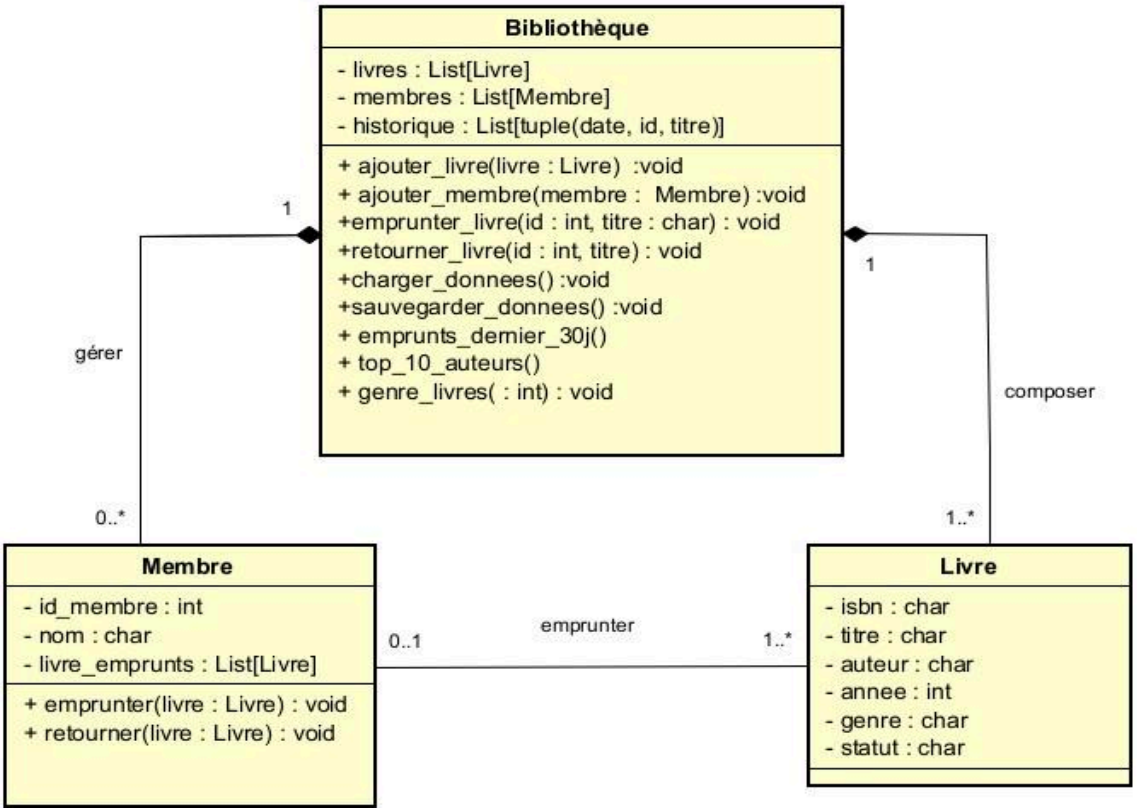
- `isbn : char` → Identifiant unique du livre.
- `titre : char`→ Titre du livre.
- `auteur : char`→ Nom de l'auteur.
- `annee : int` → Année de publication.
- `genre : char`→ Catégorie du livre.
- `statut : char`→ État du livre : "disponible" ou "emprunté".

Relations entre les Classes:

- **Bibliothèque** → **Livre** : **Composition (1 →)*
  ➤ Une bibliothèque contient plusieurs livres.
  ➤ Cycle de vie : si la bibliothèque est supprimée, les livres le sont aussi.

- **Bibliothèque** → **Membre** : **Composition (1 →)*
  ➤ Une bibliothèque gère plusieurs membres.
  ➤ Cycle de vie : si la bibliothèque est supprimée, les membres sont aussi supprimés.

- **Membre ↔ Livre** : **Association**
  ➤ Un membre peut emprunter plusieurs livres (`livres_empruntes : List[Livre]`).
  ➤ Un livre peut être emprunté par 0 ou 1 membre à la fois (indiqué par le `statut`).



IV. Explication des algorithmes clés :

1. Gestion des emprunts et retours:

Objectif :

Permettre à un membre d'emprunter un livre si celui-ci est disponible, et de le retourner plus tard.

- On cherche le membre et le livre dans les listes.
- On vérifie que le livre est disponible.
- On ajoute le livre à la liste des livres empruntés du membre.
- On change le statut du livre à "emprunté".
- On enregistre l'action dans l'historique.

2. Sauvegarde et chargement des données:

Objectif :

Stocker les livres, membres et historique dans des fichiers `.txt` ou `.csv` pour conserver les données entre les sessions.

Exemple : sauvegarde des livres

- Ouvre un fichier texte en mode écriture.
- Parcourt la liste des livres.
- Écrit chaque livre sur une ligne sous forme de texte, séparé par des points-virgules.

3. Génération des visualisations statistiques:

Objectif :

Analyser les données de l'historique et produire des graphiques exploitables.

Exemple : top 10 auteurs

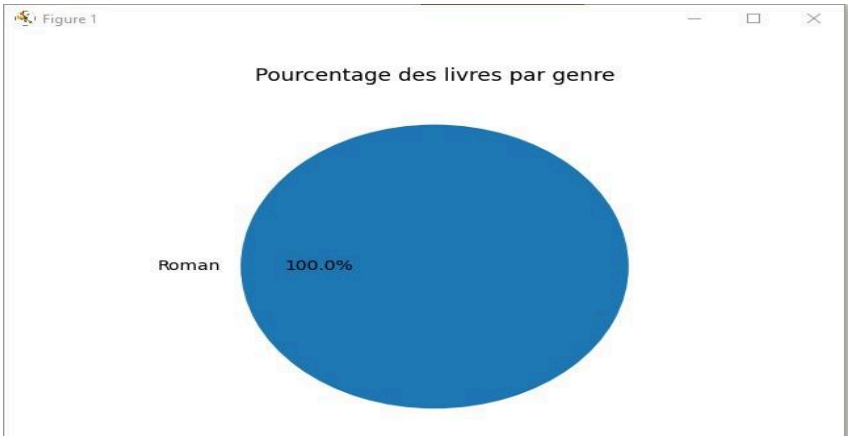
- On lit le fichier historique avec `pandas`.
- On compte les occurrences de chaque auteur avec `value_counts()`.
- On sélectionne les 10 plus grands avec `nlargest(10)`.
- On trace un graphique à barres avec `matplotlib`.

V. Visualisation des données:

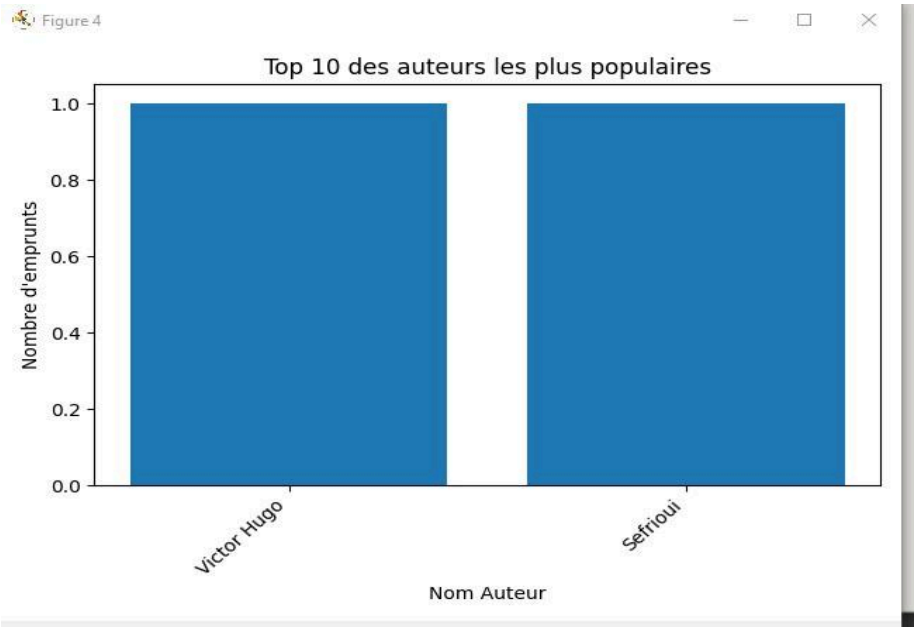
L'application propose trois types de visualisations statistiques générées dynamiquement à partir de l'historique des emprunts :

1. Répartition des livres par genre:

Ce diagramme circulaire permet de visualiser les catégories les plus représentées dans la bibliothèque.

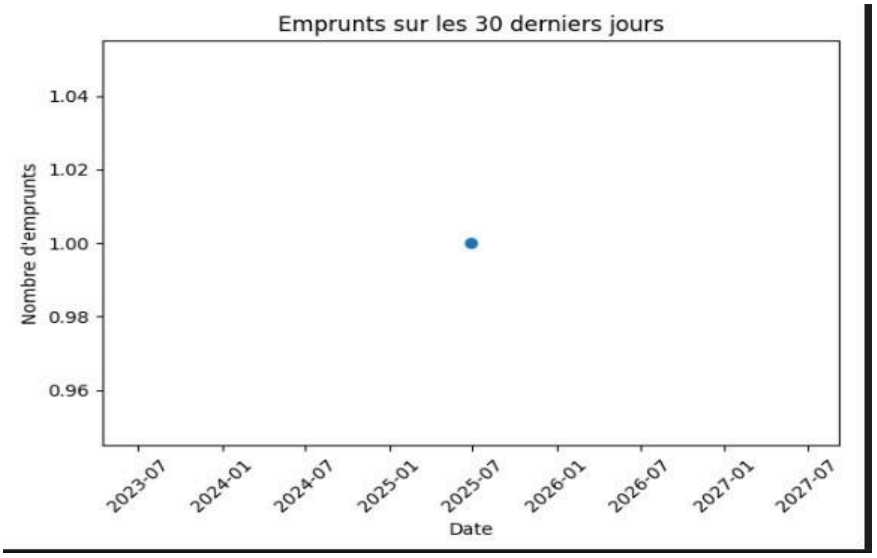


2. Top 10 des auteurs les plus empruntés:



Ce graphique en barres montre les auteurs les plus populaires auprès des membres.

3. Emprunts des 30 derniers jours:



Cette courbe retrace le nombre d'emprunts par jour, utile pour observer l'activité de la bibliothèque.

VI. Interface graphique:

L'application est conçue avec Tkinter et organisée en plusieurs onglets pour simplifier la navigation :

- **Livres** : permet d'ajouter des livres et d'afficher la liste avec leur statut (disponible ou emprunté).

- **Membres** : permet d'inscrire des membres et de visualiser la liste des inscrits.

- **Emprunts** : offre des formulaires pour emprunter et retourner des livres, et un tableau pour suivre les emprunts en cours.

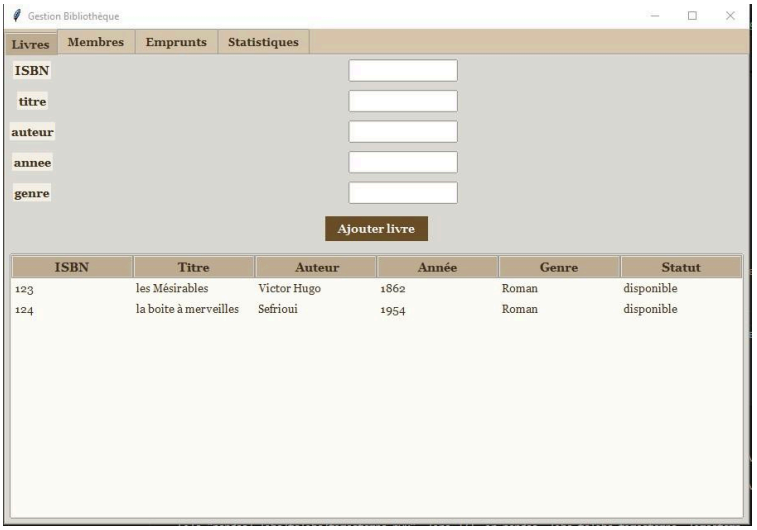
- **Statistiques** : affiche des graphiques générés avec Matplotlib (répartition par genre, top auteurs, emprunts sur 30 jours) et propose des options de sauvegarde et chargement des données.

Les onglets contiennent des champs de saisie, des boutons et des tableaux configurés pour une utilisation claire et ergonomique. Le style choisi assure une interface agréable et fonctionnelle.

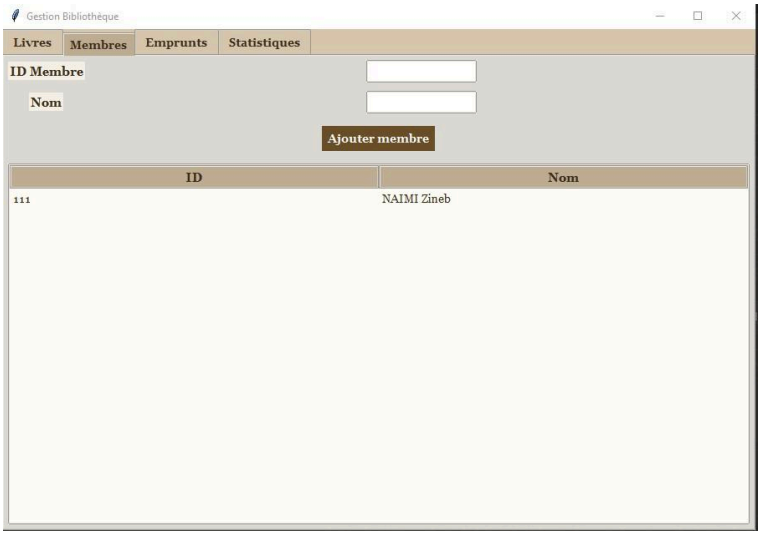
Captures d'écran:

Voici quelques captures d'écran illustrant l'interface :

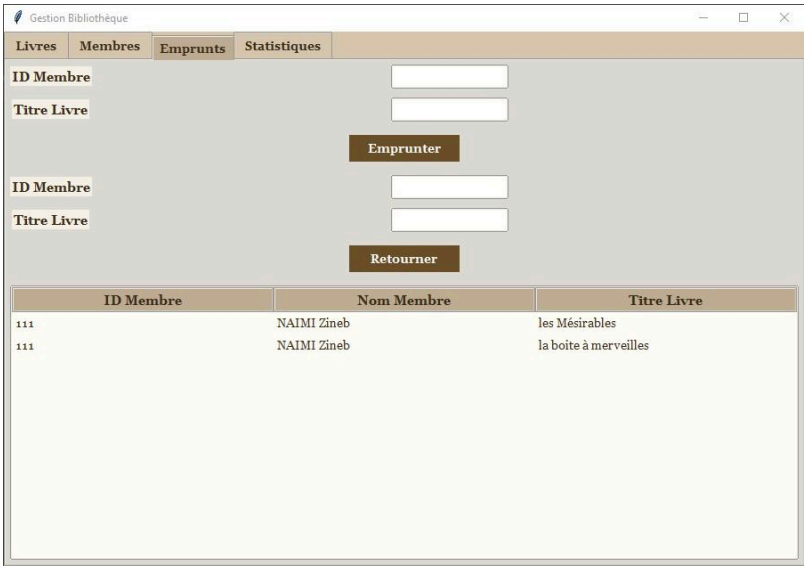
- Onglet Livres avec formulaire et liste des livres.



- Onglet Emprunts montrant les opérations d'emprunt et de retour.



- Graphiques des statistiques affichés dans l'onglet dédié.



VII. Difficultés rencontrées et solutions apportées:

Durant le développement de ce projet, plusieurs difficultés ont été rencontrées, principalement liées à la découverte et à l'utilisation de bibliothèques Python nouvelles pour moi, telles que **Tkinter**, **Matplotlib** et **Pandas**. Apprendre à utiliser ces bibliothèques dans un laps de temps réduit a représenté un vrai défi, notamment pour :

- La création d'interfaces graphiques avec **Tkinter**, qui nécessite une bonne organisation du code et une compréhension claire des événements et des widgets ;
- La manipulation et le traitement des données avec **Pandas**, en particulier pour les statistiques et le filtrage de données temporelles ;
- La génération de graphiques lisibles avec **Matplotlib**, tout en gérant les erreurs liées aux types de données ou au format des axes.

Ces difficultés ont été surmontées progressivement grâce à :

- La consultation de la documentation officielle de chaque bibliothèque ;
- L'analyse des messages d'erreurs et des tests réguliers sur de petits morceaux de code ;
- Et surtout, la persévérance et l'expérimentation active, en testant différentes approches pour obtenir le comportement attendu.

Ces obstacles ont été une excellente opportunité d'apprentissage, m'ayant permis de développer mes compétences en résolution de problèmes et en autonomie dans l'exploration de nouvelles technologies.

VIII. Conclusion :

Le développement de cette application de gestion de bibliothèque a été une expérience enrichissante sur le plan technique et personnel. Il m'a permis d'approfondir mes connaissances en programmation orientée objet, en manipulation de fichiers, en visualisation de données ainsi qu'en conception d'interfaces graphiques. En relevant les défis liés à l'intégration de plusieurs bibliothèques, j'ai acquis de nouvelles compétences et gagné en autonomie.

Le projet est fonctionnel et modulable, et pourrait évoluer à l'avenir vers une version en réseau ou avec base de données, pour répondre à des besoins plus complexes.

Ce projet m'a non seulement permis de mieux comprendre les exigences d'un logiciel de gestion réel, mais aussi d'apprendre à structurer un projet complet du début à la fin.